

Hadoop and Kerberos: The madness beyond the gate

Authors:

S. A. Loughran

Word count: 4,922

Introduction

When HP Lovecraft wrote his books about forbidden knowledge which would reduce the reader to insanity, of "Elder Gods" to whom all of humanity were a passing inconvenience, most people assumed that he was making up a fantasy world.

In fact he was documenting Kerberos.

What is remarkable is that he did this fifty years before kerberos was developed. This makes him less of an author, instead: a prophet.

What he wrote was true: there are some things humanity was not meant to know. Most people are better off living lives of naive innocence, never having to see an error message about SASL or GSS, never fear building up scripts of incantations to "kadmin.local", incantations which you hope to keep evil and chaos away. To never stare in dismay at the code whose true name must never be spoken, but instead it's initials whispered, "UGI". For those of us who have done all this, our lives are forever ruined. From now on we will cherish any interaction with a secure Hadoop cluster -from a client application to HDFS, or application launch on a YARN cluster, and simply viewing a web page in a locked down web UI -all as a miracle against the odds, against the forces of chaos struggling to destroy order.

And forever more, we shall fear those voices calling out to us in the night, the machines by our bed talking to us, saying things like "we have an urgent support call related to REST clients on a remote kerberos cluster -can you help?"

This documents contains the notes from previous people who have delved too deep into the mysteries of Hadoop and Kerberos, who have read the forbidden source code, maybe who have even contributed to it. If you wish to preserve your innocence, to view the world as a place of happiness: stop now.

Disclaimer

This document is a collection of notes based on the experience of the author. There are no guarantees that any of the information contained within was correct at the time of writing, let alone the time of reading. The author does not accept any responsibility for actions made on the basis of the information contained herein, be it correct or or incorrect.

The reader of this document is likely to leave with some basic realisation that Kerberos, while important, is an uncontrolled force of suffering and devastation. The author does not accept any responsibility for the consequences of such knowledge.

Foundational Concepts

What is the problem that Hadoop security is trying to address?

Apache Hadoop is "an OS for data". A Hadoop cluster can rapidly become the largest stores of data in an organisation. That data can explicitly include sensitive information: financial, personal, business, and can often implicitly contain data which needs to be sensitive about the privacy of individuals (for example, log data of web accesses alone). Much of this data is protected by laws of different countries. This means that access to the data needs to be strictly controlled, and accesses made of that data potentially logged to provide an audit trail of use.

You have to also consider, "why do people have Hadoop clusters?". It's not just because they have lots of data --its because they want to make use of it. A data-driven organisation needs to trust that data, or at least be confident of its origins. Allowing entities to tamper with that data is dangerous.

For the protection of data, then, read and write access to data stored directly in the HDFS filesystem needs to be protected. Applications which work with their data in HDFS also need to have their accesses restricted: Apache HBase and Apache Accumulo store their data in HDFS, Apache Hive submits SQL queries to HDFS-stored data, etc. All these accesses need to be secured; applications like HBase and Accumulo granted restricted access to their data, and themselves securing and authenticating communications with their clients.

YARN allows arbitrary applications to be deployed within a Hadoop cluster. This needs to be done without granting open access to the entire cluster from those user-launched applications, while isolating different users' work. A YARN application started by user Alice should not be able to directly manipulate an application launched by user "Bob", even if they are running on the same host. This means that not only do they need to run as different users on the same host (or in some isolated virtual/container), the applications written by Alice and Bob themselves need to be secure. In particular, any web UI or IPC service they instantiate needs to have its access restricted to trusted users. here Alice and Bob

Authentication

Authorization

Encryption

Auditing

The Limits of Hadoop Security

What are the limits of Hadoop security? Even with Kerberos enabled, what vulnerabilities exist?

Unpatched and 0-day holes in the layers underneath.

The underlying OS in a Hadoop cluster may have known or 0-day security holes, allowing a malicious (YARN?) application to gain root access to a host in the cluster. Once this is done it would have direct access to blocks stored by the datanode, and to secrets held in the various processes, including keytabs in the local filesystems.

Defences

1. Keep up to date with security issues. (SANS is worth tracking), and keep servers up to date.
2. Isolate the Hadoop cluster from the rest of your network infrastructure, apart from some "edge" nodes, so that only processes running in the cluster.
3. Developers: ensure that your code works with the more up to date versions of operating systems, JDKs and dependent libraries, so that you not holding back the upgrades. Do not increase the risk for the operations team.

Failure of users to keep their machines secure

The perennial problem. Securing end-user machines is beyond the scope of the Hadoop project.

However, one area where Hadoop may impose risk on the end-user systems is the use of Java as the runtime for client-side code, so mandating an installation of the JVM on those users who need to directly talk to the Hadoop services.

Ops teams should

- * Make sure that an up to date JVM/JRE is installed, out of date ones are uninstalled, and that Java Applets in browsers are completely disabled.
- * Control access to those Hadoop clusters and the services deployed on them.
- * Use HDFS Quotas and YARN Queues to limit the resources malicious code can do.
- * Collect the HDFS audit logs and learn how to use them to see if, after any possible security breach, you are in a position to even state what data was accessed by a specific user in a given time

period.

We Hadoop developers need to

1. Make sure that our code works with current versions of Java, and test against forthcoming releases (a permanent trouble spot).
2. Make sure that our own systems are not vulnerable due to the tools installed locally.
3. Work to enable thin-client access to services, through REST APIs over Hadoop IPC and other Java protocols, and by helping the native-client work.
4. Ensure our applications do not blindly trust users -and do as much as possible to prevent privilege escalation.
5. Log information for ops teams to use in security audits.

Denial of service attacks.

Hadoop is its own Distributed Denial of Service platform. A misconfiguration could easily trigger all datanodes to attempt to report in so frequently that the namenode gets overloaded, triggering apparent timeouts of some DN heartbeats, leading to the namenode assuming it has failed and starting block transfers of under-replicated blocks, so impacting network load and reporting even more. This is not a hypothetical example: Facebook had a cluster outage from precisely such an event, a failing switch partitioning the cluster and triggering a cascade failure. Nowadays IPC throttling (from Twitter) and the use of different ports on the namenode for heartbeating and filesystem operations (from Facebook) try to keep this under control.

We're not aware of any reported deliberate attempts to use a Hadoop cluster to overload local/remote services, though there are some anecdotes of the Yahoo! search engines having been written so as to deliberately stripe searches not just across hosts, but domains and countries, so as not to overload the DNS infrastructure of small countries. If you have some network service in your organisation which is considered critical (Examples: sharepoint, exchange), then configure the firewall rules to block access to those hosts and service ports from the Hadoop cluster.

Other examples of risk points and mitigation strategies

YARN resource overload

Too many applications asking for small numbers of containers, consuming resources in the Node Managers and RM. There are minimum size values for YARN container allocations for a reason: it's good to set them low on a single node development VM, but in production, they are needed

DNS overload.

This is easily done by accident. Many of the large clusters have local caching DNS servers for this reason, especially those doing any form of search.

CPU, network IO, disk IO, memory

YARN applications can consume so much local resources that they hurt the performance of other applications running on the same nodes.

In Linux and Windows, CPU can be throttled, the amount of physical and virtual memory limited. We could restrict disk and network IO (see relevant JIRAs), but that won't limit HDFS IO, which takes place in a different process.

YARN labels do let you isolate parts of the cluster, so that low-latency YARN applications have access to machines across the racks which IO-heavy batch/background applications do not.

Deliberate insertion of malicious code into the Hadoop stack, dependent components or underlying OS.

We haven't encountered this yet. Is it conceivable? Yes: in the security interfaces and protocols themselves. Anything involving encryption protocols, random number generation and authentication checks would be the areas most appealing as targets: break the authentication or weaken the encryption and data in a Hadoop cluster becomes more accessible. As stated, we've not seen this. As Hadoop relies on external libraries for encryption, we have to trust them (and any hardware implementations), leaving random number generation and authentication code as targets. Given that few committers understand Hadoop Kerberos, especially at the REST/SPNEGO layer, it is hard for new code submissions in this area to be audited well.

One risk we have to consider is: if someone malicious had access to the committer credentials of a developer, could they insert malicious code? Everyone in the Hadoop team would notice changes in the code appearing without associated JIRA entries, though it's not clear how well reviewed the code is.

Mitigation strategies. A key one has to be "identify those areas which would be vulnerable to deliberate weakening, and audit patch submissions extra rigorously there", "reject anything which appears to weaken security -even something as simple as allowing IP addresses instead of Hostnames in kerberos binding (cite: JIRA) could be dangerous. And while the submitters are probably well-meaning, we should assume maliciousness or incompetence in the high-risk areas. (* yes, this applies to my own patches too. The accusation of incompetence is defensible based on past submissions anyway).

Insecure applications

SQL injection attacks are the classic example here. It doesn't matter how secure the layers are underneath if the front end application isn't handling untrusted data. Then there are things like emergency patches to apple watches because of a binary parse error in fonts.

Mitigation strategies

1. assume all incoming data is untrusted. In particular, all strings used in queries, while all documents (XML, HTML, binary) should be treated as potentially malformed, if not actually malicious.
2. Use source code auditing tools such as Coverity Scan to audit the code. Apache projects have free access to some of these tools.
3. Never have your programs ask for more rights than they need, to data, to database tables (and in HBase and Accumulo: columns)
4. Log data in a form which can be used for audit logs. (Issue: what is our story here? Logging to local/remote filesystems isn't it, not if malware could overwrite the logs)

Hadoop IPC Security

Adding a new IPC interface to a Hadoop Service/ Application

This is "fiddly". It's not impossible, it just involves effort. In its favour: it's a lot easier than SPNEGO.

SecurityInfo

Every exported RPC service will need its own extension of the SecurityInfo class, to provide two things

1. The name of the principal to use in this communication
1. The token used to authenticate ongoing communications.

PolicyProvider

A PolicyProvider subclass. This is used to inform the RPC infrastructure of the ACL policy: who may talk to the service. It must be explicitly passed to the RPC server

```
rpcService.getServer().refreshServiceAcl(serviceConf,  
    new MyRPCPolicyProvider());
```

SecurityInfo

The resource file META-INF/services/
org.apache.hadoop.security.SecurityInfo lists all RPC APIs
which have a matching SecurityInfo subclass in that JAR.

```
# Licensed to the Apache Software Foundation (ASF) under one or  
# more  
# contributor license agreements. See the NOTICE file  
# distributed with  
# this work for additional information regarding copyright  
# ownership.  
# The ASF licenses this file to You under the Apache License,  
# Version 2.0  
# (the "License"); you may not use this file except in  
# compliance with  
# the License. You may obtain a copy of the License at  
#  
# http://www.apache.org/licenses/LICENSE-2.0  
#  
# Unless required by applicable law or agreed to in writing,  
# software
```



```
# distributed under the License is distributed on an "AS IS"  
BASIS,  
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express  
or implied.  
# See the License for the specific language governing  
permissions and  
# limitations under the License.  
org.apache.example.appmaster.rpc.RPCSecurityInfo
```

The RPC framework will read this file and build up the security information for the APIs (server side? Client side? both?)

UGI

If there is one class guaranteed to strike fear into anyone with experience in Hadoop+Kerberos code it is UserGroupInformation, abbreviated to "UGI"

UGI troublespots

- * It's a singleton. Don't expect to have one "real user" per process. This sort of makes sense when you think about it.

- * Once initialized, it stays initialized *and cannot be reset*. This makes it critical to load in your configuration information including keytabs and principals, before that first initialization of the UGI.

- * UGI init can take place in code which you don't expect. A specific example is in the Hadoop filesystem APIs. Create Hadoop filesystem instances and UGI is likely to be init'd immediately, even if it is a local file:// reference. As a result: init before you go near the filesystem, with the principal you want.

- * All its exceptions are basic IOExceptions, so hard to match on without looking at the text, which is very brittle.

- * Some invoked operations are relayed without the stack trace (this should now be fixed).

- * Diagnostics could be improved. (this is one of those British understatements, it really means "it would be really nice if you could actually get any hint as to WTF is going inside the class as otherwise you are left with nothing to go on other than some message that a user at a random bit of code wasn't authorized")

The issues related to diagnostics, logging, exception types and inner causes could be addressed. It would be nice to also have an exception cached at init time, so that diagnostics code could even track down where the init took place. Volunteers welcome. That said, here are some bits of the code where patches would be vetoed

- The text of exceptions. We don't know what is scanning for that text, or what documents go with them.

- All exceptions must be subclasses of IOException.

- Logging must not leak secrets, such as tokens.

SPNEGO

SPNEGO is the acronym of the protocol by which HTTP clients can authenticate with a web site using Kerberos. This allows the client to identify and authenticate itself to a web site or a web service.

SPNEGO is supported by

- * the standard browsers, to different levels of pain of use
- * ``curl`` on the command line
- * ``java.net.URL`` in Java7+

The final point is key: it can be used programmatically in Java, so used by REST client applications to authenticate with a remote Web Service.

CAUTION

Apache Http Components do not support SPNEGO. As the documentation says "try it and see" {cite}.

Exactly how the Java runtime implements its SPNEGO authentication is a mystery to all. Unlike, say Hadoop IPC, where the entire authentication code has been implemented by people whose email addresses you can identify from the change log and so ask hard questions, what the JDK does is a black hole.

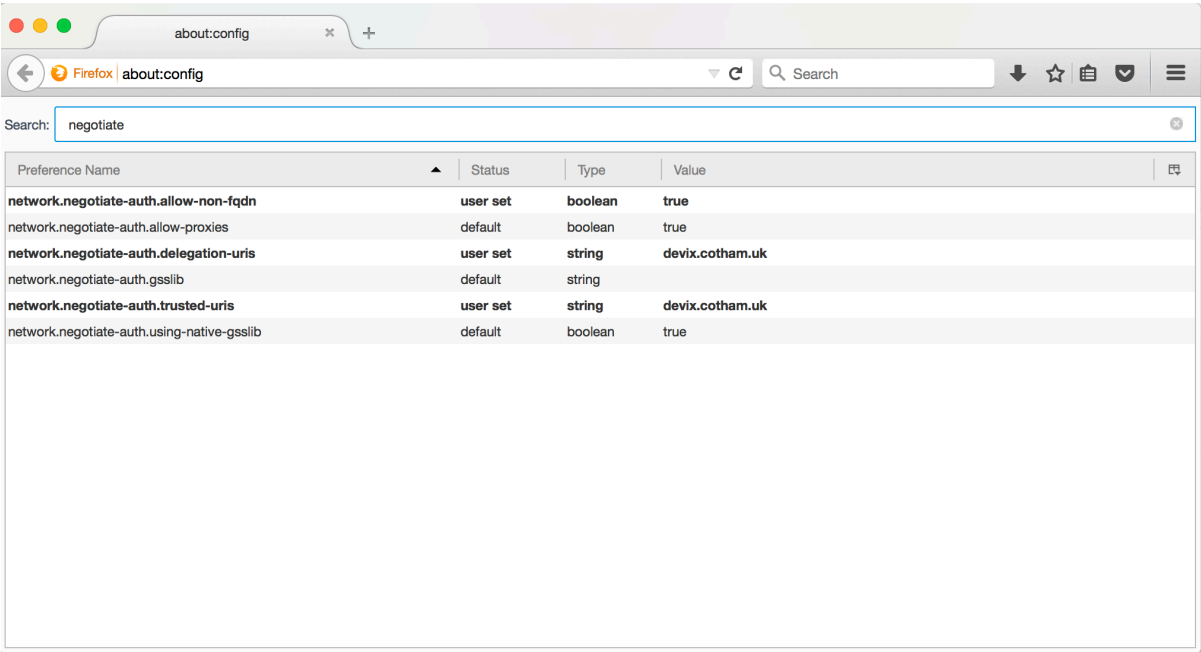
Configuring Firefox to use SPNEGO

Firefox is the easiest browser to set up with SPNEGO support, as it is done in `about:config` and then persisted

Here are the settings for a local VM, a VM which has an entry in the `/etc/hosts`:

```
192.168.1.134 devix.cotham.uk devix
```

This hostname is then listed in firefox's config as a URL to trust.



ZOOKEEPER and SASL

Apache Zookeeper uses SASL to authenticate callers.

Other than SASL, its access control is all based around secrets which are shared between client and server, and sent over the (unencrypted) channel. This means they cannot be relied upon to securely identify callers.

Enabling SASL in ZK

SASL is enabled in ZK by setting a system property. While adequate for a server, it's less than convenient when using ZK in an application as it means something very important: you cannot have a non-SASL and a SASL ZK connection at the same time. (you could create one connection, change the system properties and then create the next, but as Apache Curator doesn't do this, and everyone sensible uses Curator to handle transient disconnections and ZK node failover, this isn't practicable). Someone needs to fix this.

JVM kerberos config customisation

java.security.krb5.conf

The JVM kerberos operations are configured via krb5.conf file specified in the JVM option "java.security.krb5.conf", which can be done on the JVM command line, or inside the JVM

```
System.setProperty("java.security.krb5.conf", krbfilepath);
```

Notes

- * use double backslash to escape paths on Windows platforms, e.g. C:\\keys\\key1, or \\\\server4\\shared\\tokens

- * Different JVMs (e.g. IBM JVM) want different fields in their krb5.conf file. How can you tell? Kerberos will fail with a message

- * the JVM property MUST be set before UGI is initialized

java.security.krb5.realm

This system property sets the realm for the kerberos binding. This allows you to use a different one from the default in the krb5.conf file.

Example

```
-Djava.security.krb5.realm=PRODUCTION
```

FAQ

Do I need to specify a krb conf file?

-no, not if there is one in /etc/krb5.conf or wherever else your default OS keeps one *and those values are what you want*

What are those custom fields you mention?

-I don't remember. I recall that some had to be set up when doing the MiniKDC tests (which do need a custom krb5.conf file),

When should these properties be set

As soon as possible in JVM startup. Certainly before UGI initialization.

JAAS Configuration

Logging

Hadoop Logging

JVM Kerberos Library logging

You can turn Kerberos low-level logging on

```
-Dsun.security.krb5.debug=true
```

This doesn't come out via Log4J, or java.util logging; it just comes out on the console. Which is somewhat inconvenient -but bear in mind they are logging at a very low level part of the system. And it does at least log.

If you find yourself down at this level you are in trouble. Bear that in mind.

Error Messages to Fear

OS/JVM Layer

Some of these are covered in Oracle's Troubleshooting Kerberos docs. This section just highlights some of the common causes, other causes that Oracle don't mention -and messages they haven't covered.

Server not found in Kerberos database (7)

- * DNS is a mess and your machine does not know its own name.
- * Your machine has a hostname, but it's not one there's an entry in the keytab for

No valid credentials provided (Mechanism level: Illegal key size)]

Your JVM doesn't have the extended cryptography package and can't talk to the KDC. Switch to openjdk or go to your JVM supplier (Oracle, IBM) and download the JCE extension package, and install it in the hosts where you want Kerberos to work.

No valid credentials provided (Mechanism level: Failed to find any Kerberos tgt

This may appear in a stack trace starting with something like:

```
javax.security.sasl.SaslException: GSS initiate failed [Caused  
by GSSException: No valid credentials provided (Mechanism level:  
Failed to find any Kerberos tgt)]
```

Causes

1. You aren't logged in via `kinit`.
2. You did specify a keytab but it isn't there or is somehow otherwise invalid
3. You don't have the Java Cryptography Extensions installed.

Clock skew too great

```
## GSSException: No valid credentials provided (Mechanism level:
Attempt to obtain new INITIATE credentials failed! (null)) . . .
Caused by: javax.security.auth.login.LoginException: Clock skew
too great
```

This comes from the clocks on the machines being too far out of sync. This can surface if you are doing Hadoop work on some VMs and have been suspending and resuming them; they've lost track of when they are. Reboot them.

If its a physical cluster, make sure that your NTP daemons are pointing at the same NTP server, one that is actually reachable from the Hadoop cluster. And that the timezone settings of all the hosts are consistent.

KDC has no support for encryption type

This crops up on the MiniKDC if you are trying to be clever about encryption types. It doesn't support many.

```
## Failure unspecified at GSS-API level (Mechanism level:
Checksum failed)
```

Kerberos is very strict about hostnames and DNS <http://stackoverflow.com/questions/12229658/java-spnego-unwanted-spn-canonicalization>; Java 8 behaves differently from Java 6 & 7 here which can cause problems (HADOOP-11628)[<https://issues.apache.org/jira/browse/HADOOP-11628>].

Hadoop Web/REST APIs

AuthenticationToken ignored

Surfaces in the HTTP logs of Hadoop REST/Web UIs:

```
2015-06-26 13:49:02,239 WARN
org.apache.hadoop.security.authentication.server.AuthenticationF
ilter: AuthenticationToken ignored:
org.apache.hadoop.security.authentication.util.SignerException:
Invalid signature
```

This means that the caller did not have the credentials to talk to a Kerberos-secured channel.

1. The caller may not be logged in.
2. The caller may have been logged in, but its kerberos token

has expired, so its authentication headers are not considered valid any more.

3. The time limit of a negotiated token for the HTTP connection has expired. Here the calling app is expected to recognise this, discard its old token and renegotiate a new one. If the calling app is a YARN hosted service, then something should have been refreshing the tokens for you.

Writing Kerberos Tests with MiniKDC

The Hadoop project has an in-VM Kerberos Controller for tests, MiniKDC, which is packaged as its own JAR for downstream use. The core of this code actually comes from the Apache Directory Services project.

Testing against Kerberized Hadoop clusters

This is not actually the hardest form of testing; getting the MiniKDC working holds that honour.

It does have some pre-requisites

1. Everyone running the tests has set up a Hadoop cluster/single VM with Kerberos enabled.
2. The software project has a test runner capable of deploying applications into a remote Hadoop cluster/VM and assessing the outcome.

It's primarily the test runner which matters. Without that you cannot do functional tests against any Hadoop cluster.

However, once you have such a test runner, you have a powerful tool: the ability to run tests against real Hadoop clusters, rather than simply minicluster and miniKDC tests which, while better than nothing, are unrealistic.

If this approach is so powerful, why not bypass the minicluster tests altogether?

1. Minicluster tests are easier to run. Build tools can run them; Jenkins can trivially run them as part of test runs.
2. The current state of the cluster affects the outcome of the tests. Its useful not only to have tests tear down properly, but for the setup phase of each test suite to verify that the cluster is in the valid initial state/get it into that state. For YARN applications, this generally means that there are no running applications in the cluster.
3. Setup often includes the overhead of copying files into HDFS. As the secure user.
4. The host launching the tests needs to be setup with kinit/keytabs.
5. Retrieving and interpreting the results is harder. Often it involved manually going to the YARN RM to get through to the logs (assuming that yarn-site is configured to preserve them), and/or collecting other service logs.
6. If you are working with nightly builds of Hadoop, VM setup needs to be automated.
7. Unless you can mandate and verify that all developers run the tests against secure clusters, they may not get run by everyone.

8. The tests can be slow.
9. Fault injection can be harder.

Overall, tests are less deterministic.

In the slider project, different team members have different test clusters, Linux and Windows, Kerberized and non-Kerberized, Java-7 and Java 8. This means that test runs do test a wide set of configurations without requiring every developer to have a VM of every form. The Hortonworks QE team also run these tests against the nightly HDP stack builds, catching regressions in both the HDP stack and in the Slider project.

For fault injection the Slider Application Master has an integral "chaos monkey" which can be configured to start after a defined period of time, then randomly kill worker containers and/or the application master. This is used in conjunction with the functional tests of the deployed applications to verify that they remain resilient to failure. When tests do fail, we are left with the problem of retrieving the logs and identifying problems from them. The QE test runs do collect all the logs from all the services across the test clusters - but this still leaves the problem of trying to correlate events from the logs across the machines.

Low-level secrets

KRB5CCNAME

The environment variable `KRB5CCNAME` (cite: <http://web.mit.edu/kerberos/krb5-1.4/krb5-1.4/doc/klist.html>)

As the docs say:

If the KRB5CCNAME environment variable is set, its value is used to name the default ticket cache.

Notes of Obscurity

IP addresses vs. Hostnames

Kerberos principals are traditionally defined with hostnames of the form `hbase@worker3/EXAMPLE.COM`, not `hbase/10.10.15.1/EXAMPLE.COM`

The issue of whether Hadoop should support IP addresses has been raised [HADOOP-9019](<https://issues.apache.org/jira/browse/HADOOP-9019>) [HADOOP-7510](<https://issues.apache.org/jira/browse/HADOOP-7510>)

Current consensus is no: you need DNS set up, or at least a consistent and valid `/etc/hosts` file on every node in the cluster.

Warning: Windows does not reverse-DNS `127.0.0.1` to `localhost` or the local machine name; this can cause problems with MiniKDC tests in Windows, where adding a `user/127.0.0.1@REALM` principal will be needed [example] (<https://github.com/apache/hadoop/blob/trunk/hadoop-yarn-project/hadoop-yarn/hadoop-yarn-registry/src/test/java/org/apache/hadoop/registry/secure/AbstractSecureRegistryTest.java#L209>).

Checklists

Hadoop RPC Service

[] Principal for Service defined. This is generally a configuration property.

[] `SecurityInfo` subclass written.

[] `META-INF/services/org.apache.hadoop.security.SecurityInfo` resource lists.

the `SecurityInfo` subclass.

[] `PolicyProvider` subclass written.

[] RPC server handed `PolicyProvider` subclass during setup.

YARN Client/launcher

[] `HADOOP\_USER` env variable set on AM launch context in insecure clusters.

[] In secure cluster: all delegation tokens needed (HDFS, Hive, HBase, Zookeeper) created and added to launch context.

YARN Application

[] Delegation tokens extracted and saved.

[] When launching containers, the relevant subset of delegation tokens are passed to the containers. (This normally omits the RM/AM token).

[] Container Credentials are retrieved in AM and containers.

Web Service

[] `AuthenticationFilter` added to web filter chain

[] Token renewal policy defined and implemented.

(Look at `TimelineClientImpl` for an example of this)

Clients

All clients

- [] Supports keytab login and calls `UserGroupInformation.loginUserFromKeytab(principalName, keytabFilename)` call during initialization.

- [] Issues `UserGroupInformation.getCurrentUser().checkTGTAndReloginFromKeytab()` call during connection setup/token reset. This is harmless on an insecure or non-keytab client.

- [] Client supports Authentication Token option

- [] Client supports Delegation Token option. (not so relevant for most YARN clients)

- [] For Delegation-token authenticated connections, something runs in the background to regularly update delegation tokens.

- [] Tested against secure clusters with user logged out (`kdestroy`).

- [] Logs basic security operations at INFO, with detailed operations at DEBUG level.

RESTful client

- [] Jersey: URL constructor handles SPNEGO Auth

- [] URL constructor reacts to 401/403 responses when using Authentication Token by deleting creating a new Auth Token and re-issuing request. (this triggers re-authentication)

References

[Leave this empty to use a reference manager like Zotero. Once you create a bibliography in LibreOffice/Word, paste the generated references into this part of the document]

1. [Adding Security to Apache Hadoop](http://hortonworks.com/wp-content/uploads/2011/10/security-design_withCover-1.pdf)
1. [The Role of Delegation Tokens in Apache Hadoop Security](<http://hortonworks.com/blog/the-role-of-delegation-tokens-in-apache-hadoop-security/>)
1. [Chapter 8. Secure Apache HBase](<http://hbase.apache.org/book/security.html>)
1. Hadoop Operations p135+
1. [Java 7 Kerberos Requirements](<http://docs.oracle.com/javase/7/docs/technotes/guides/security/jgss/tutorials/KerberosReq.html>)
1. [Java 8 Kerberos Requirements](<http://docs.oracle.com/javase/8/docs/technotes/guides/security/jgss/tutorials/KerberosReq.html>)
1. [Troubleshooting Kerberos on Java 7](<http://docs.oracle.com/javase/7/docs/technotes/guides/security/jgss/tutorials/Troubleshooting.html>)
1. [Troubleshooting Kerberos on Java 8](<http://docs.oracle.com/javase/8/docs/technotes/guides/security/jgss/tutorials/Troubleshooting.html>)
1. [JAAS Configuration (Java 8)](<http://docs.oracle.com/javase/8/docs/technotes/guides/security/jgss/tutorials/LoginConfigFile.html>)
1. For OS/X users, the GUI ticket viewer is ``/System/Library/CoreServices/Ticket\ Viewer.app``

